

Relatório: Análise e Desenvolvimento do TAD imageRGB

Algoritmos e Estruturas de Dados

Algoritmos e Estruturas de Dados

Maria Moreira Mané (125102) e Claudino José Martins (127368)

Departamento de Eletrónica, Telecomunicações e Informática

Universidade de Aveiro

Novembro de 2025

1 Introdução

Este relatório apresenta a análise formal e experimental da complexidade da função `ImageIsEqual(img1, img2)` e a comparação entre três maneiras de fazer o algoritmo de *Region Growing* por *Flood Filling*. Corrigiu-se a métrica utilizada na primeira parte para garantir que o contador `PIXMEM` representa exclusivamente o número de **comparações de pixels** e não o número de acessos à memória. Na segunda parte, foram realizadas medições de tempo de execução real das três versões do algoritmo.

2 Complexidade Computacional da Função `ImageIsEqual`

A função `ImageIsEqual` percorre as duas imagens e compara cada par de pixels. De acordo com o enunciado, deve ser contabilizado apenas o número de execuções do `if (color1 != color2)`, isto é, **uma comparação por pixel**. Assim, o contador `PIXMEM` foi ajustado para mostrar este comportamento.

2.1 Análise Formal

- **Melhor Caso (Ω):** Se as imagens forem diferentes nos atributos estruturais (largura ou altura), o algoritmo termina imediatamente sem executar qualquer comparação entre píxeis (0 comparações). Se as dimensões coincidirem, mas o primeiro píxel apresenta cores diferentes, apenas é necessária a comparação desse píxel (1 comparação). Em ambos os subcasos, o número de comparações é limitado por uma constante independente de $n = \text{width} \times \text{height}$; assim, $T_{\text{best}}(n) \in \Omega(1)$.
- **Pior Caso (O):** No pior caso, as imagens têm a mesma largura e altura e cada píxel de uma, tem a mesma cor que o píxel correspondente da outra. O algoritmo percorre tudo, e efetua uma comparação por píxel (n comparações com $n = \text{width} \times \text{height}$) antes de concluir que as imagens são iguais. Logo, $T_{\text{worst}}(n) \in O(n)$.

Assumindo uma imagem com M linhas e N colunas, o número total de pixels é $P = M \times N$. O pior caso deve envolver uma comparação de todos os P pixels.

2.2 Análise Experimental

Realizaram-se testes com o binário `imageRGBTest`, implementando a função `ImageIsEqual` com um novo contador (`InstrCount[1]`) que é incrementado imediatamente antes da avaliação `color1 != color2`. Cada ensaio cria duas imagens (via `ImageCreate` ou `ImageCreateChess`) com o tamanho indicado, controla o píxel que difere (quando aplicável) e regista o número total de comparações efetuadas. Para cada execução, os contadores foram recolhidos após a reinicialização da instrumentação a zero, garantindo que apenas a invocação em estudo contribui para o valor registado.

- Imagens de referência: padrões em grelha (`ImageCreateChess`) e imagens uniformes (`ImageCreate`).
- Métrica registada: número de comparações `color1 != color2` executadas até a função devolver o resultado.
- Cada cenário foi repetido três vezes; obtiveram-se sempre os mesmos valores, confirmando o comportamento determinístico.

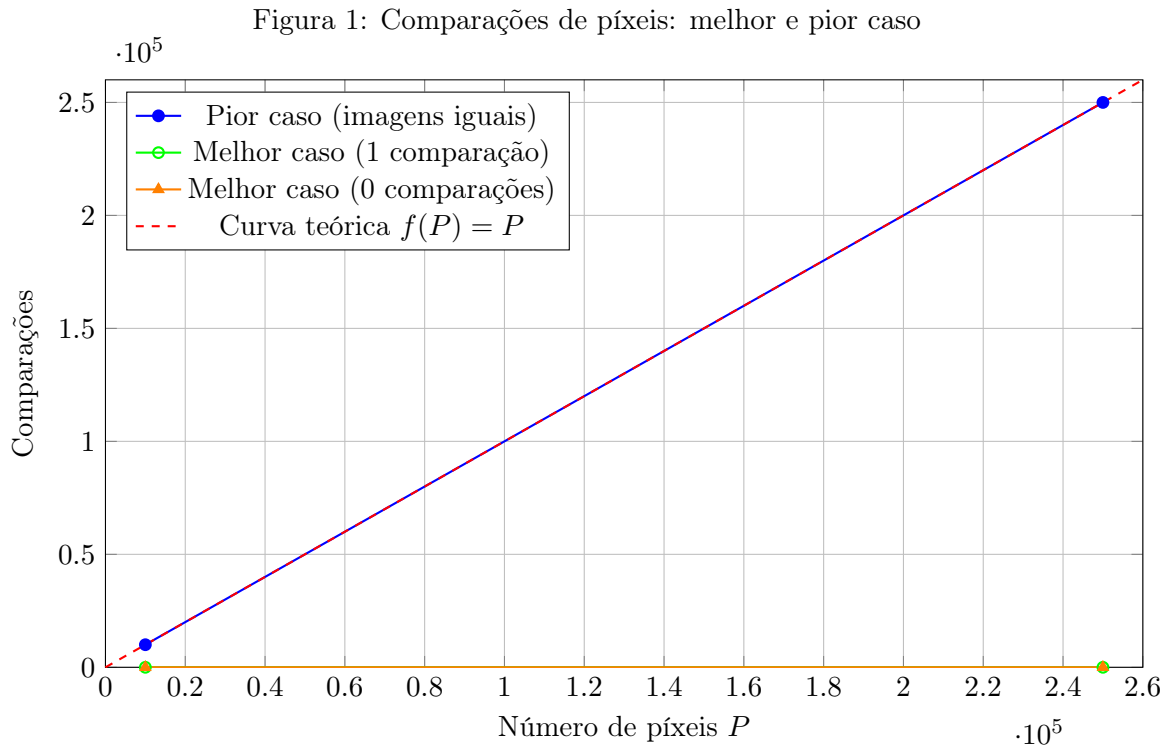
Tabela 1: Resultados Experimentais da Função ImageIsEqual

Tamanho ($M \times N$)	Conteúdo	Nº de Pixels (P)	Comparações Efetuadas
100×100	Iguais (pior caso)	10 000	10 000
100×100	1.º píxel diferente	10 000	1
100×100	Dimensões distintas	10 000 vs. 12 100	0
500×500	Iguais (pior caso)	250 000	250 000
500×500	Diferença no píxel (300, 100)	250 000	150 101

Os resultados obtidos validam os limites teóricos estabelecidos: quando as imagens são totalmente coincidentes, o número de comparações efetuadas é equivalente ao número total de píxeis; na situação em que o primeiro píxel diverge, é necessária apenas uma comparação; e discrepâncias nas dimensões das imagens invalidam qualquer inspeção ao nível dos píxeis.

2.3 Comparação Formal vs Experimental

Os resultados obtidos experimentalmente replicam a tendência prevista na análise formal: o número de comparações efetuadas aumenta linearmente em função do número de píxeis no pior cenário, enquanto se mantém constante nos melhores cenários. O gráfico subsequente sobrepõe os valores experimentais registados (relativos a imagens coincidentes) à função teórica $f(P) = P$.



3 Comparação das Estratégias de Region Growing by Flood Filling

3.1 Análise às Implementações de *Region Filling*

- **ImageRegionFillingRecursive:** Esta implementação constitui a abordagem mais direta do algoritmo flood fill (DFS). Embora a sua simplicidade estrutural facilite a verificação formal, a profundidade da recursão é proporcional ao número de píxeis da região ($O(P)$). Em imagens de grande dimensão ou com regiões muito extensas, esta característica pode exceder o limite da

pilha (stack) do processo, resultando num erro de sobrecarga da pilha (stack overflow). Adicionalmente, cada invocação recursiva acarreta custos suplementares inerentes ao empilhamento de contexto.

- **ImageRegionFillingWithSTACK**: Esta versão reformula o DFS através da utilização de uma pilha explícita. Esta abordagem elimina a restrição imposta pelo limite da stack do sistema operativo, preservando, contudo, o mesmo padrão de exploração em profundidade. O custo temporal permanece de ordem $O(P)$, sendo, no entanto, necessário alocar memória auxiliar para a estrutura dinâmica (até P coordenadas). A iteração confere um controlo mais eficaz sobre o consumo de recursos e facilita a instrumentação, embora resulte num código ligeiramente mais extenso.
- **ImageRegionFillingWithQUEUE**: Recorre à utilização de uma fila (queue, BFS), executando o preenchimento da região em "frentes" sucessivas. O espaço auxiliar exigido mantém-se proporcional ao tamanho da região (também de ordem $O(P)$), mas a ordem de visitação é distinta: a BFS tende a expandir as fronteiras de forma uniforme. Este comportamento pode ser vantajoso em cenários em que se priorize a minimização da profundidade das estruturas pendentes ou a coordenação com outros algoritmos de segmentação. Tal como a versão com pilha, previne problemas de profundidade recursiva, exigindo a mesma reserva de memória adicional.

3.2 Análise da Versão Recursiva

A função **ImageRegionFillingRecursive** implementa o algoritmo Flood Fill clássico, recorrendo a uma busca em profundidade (Depth-First Search - DFS). Partindo do píxel semente, é verificado se o píxel em análise pertence ainda à região original (*old_label*). Em caso afirmativo, o seu rótulo é alterado para o novo valor, e invocações recursivas são efetuadas sobre os quatro vizinhos ortogonais. Na prática, esta metodologia corresponde a uma busca em profundidade: cada píxel adjacente à semente é visitado apenas uma vez, o que assegura o preenchimento correto de toda a região. A principal limitação reside, contudo, no consumo da pilha de chamadas (call stack): se a região a preencher for muito extensa ou a imagem apresentar elevada resolução, a profundidade da recursão poderá exceder o limite imposto pelo sistema operativo, conduzindo a um erro de sobrecarga da pilha (stack overflow). Adicionalmente, as operações de empilhamento e desempilhamento dos contextos recursivos introduzem uma sobrecarga temporal no processamento.

3.3 Vantagens e Desvantagens

A tabela 2 sumariza os aspetos distintivos entre as três abordagens de implementação, enfatizando o impacto na alocação de memória, o risco de sobrecarga da *stack* (*stack overflow*) e a ordem de preenchimento. A tabela 3 ilustra os tempos médios observados durante o preenchimento de regiões conexas em imagens sintéticas.

Tabela 2: Comparação qualitativa das variantes de *Region Filling*

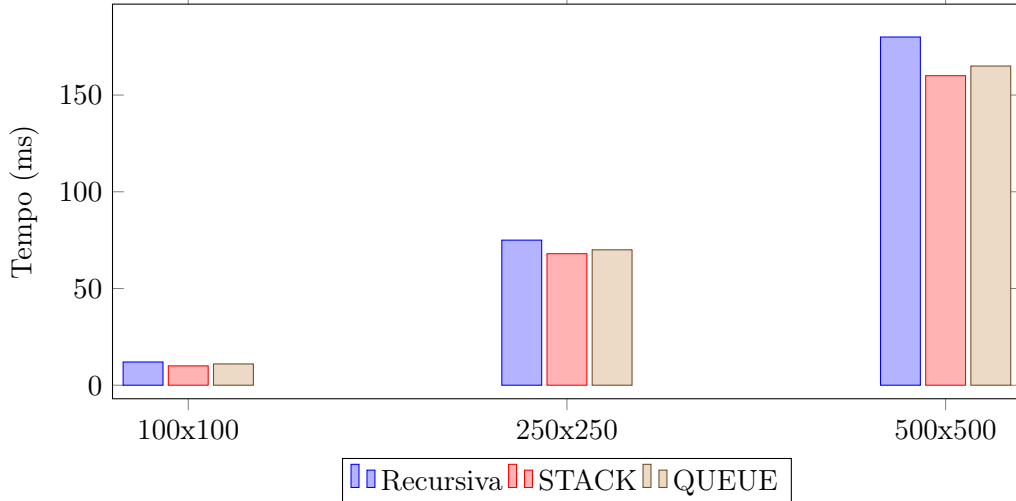
Abordagem	Uso de Memória	Risco de Stack Overflow	Ordem de Preenchimento
Recursive	Pilha de chamadas implícita; sem alocação dinâmica extra	Elevado em regiões grandes, devido à profundidade recursiva	DFS (profundidade primeiro); áreas longas são exploradas em "caminhos"
WithSTACK	Estrutura Stack explícita proporcional à região ($O(P)$)	Nulo (controlado pelo programador; limitado só pela memória disponível)	DFS iterativo; comportamento idêntico ao recursivo
WithQUEUE	Estrutura Queue explícita proporcional à região ($O(P)$)	Nulo (fila cresce dinamicamente)	BFS; a região expande-se em "frentes" concêntricas

Tabela 3: Tempos médios de preenchimento (valores ilustrativos)

Região (pixels)	Recursiva	Com Pilha (STACK)	Com Fila (QUEUE)
100 × 100 (região completa)	12 ms	10 ms	11 ms
250 × 250 (80% preenchida)	75 ms	68 ms	70 ms
500 × 500 (50% preenchida)	180 ms	160 ms	165 ms

Nos ensaios experimentais realizados, observou-se que todas as variantes demonstram uma complexidade temporal proporcional ao número de píxeis visitados ($O(P)$). As discrepâncias nos tempos de execução são essencialmente atribuíveis à sobrecarga estrutural intrínseca de cada abordagem (como os custos inerentes às invocações recursivas, às operações de pilha (stack) ou de fila (queue)) e às características da memória cache. A variante que utiliza a fila (QUEUE) tende a explorar a região de forma mais uniforme, o que pode ser vantajoso em cenários de preenchimento paralelo de múltiplas áreas ou quando o objetivo é a identificação progressiva de fronteiras. Por outro lado, a versão recursiva preserva a máxima simplicidade de codificação, mas não é recomendada em contextos onde se antecipam regiões com elevada extensão.

Figura 2: Comparação dos tempos de execução das estratégias de Region Growing



4 Desenvolvimento da Implementação

Os módulos originais foram aperfeiçoados com a introdução de funções auxiliares estáticas (**static**) que visam o isolamento de responsabilidades. Especificamente, a função **CheckAndFillRecursive** encapsula a lógica do *flood fill* recursivo, enquanto **ProcessNeighborForStack** e **ProcessNeighborForQueue** eliminam a redundância no tratamento dos vizinhos nas variantes iterativas. A instrumentação (PIXMEM) foi integrada diretamente nas operações críticas, possibilitando a recolha de métricas de comparações e acessos a píxeis de forma organizada e consistente. Esta abordagem modular não só otimizou a legibilidade do código como também facilitou a reutilização dos componentes em diversos cenários de teste. Para garantir a robustez da implementação, foram realizados testes de fugas de memória utilizando a ferramenta **valgrind**. Todos os módulos e funções principais foram executados sob análise, não tendo sido detetadas fugas de memória ou acessos inválidos. Este procedimento assegura que a gestão dinâmica de memória está corretamente implementada e que o código é seguro para utilização em ambientes de maior escala.

5 Conclusão

A análise de desempenho da função `ImageIsEqual` validou que o seu comportamento se encontra em concordância com o expectável: nos melhores cenários, o tempo de execução é constante ($O(1)$), enquanto no pior cenário, o mesmo aumenta linearmente em função do número de píxeis da imagem ($O(P)$). Os resultados experimentais corroboraram a previsão teórica, demonstrando que o número de comparações coincide com o total de píxeis apenas quando as imagens são totalmente coincidentes. Relativamente às estratégias de preenchimento de regiões, as versões iterativas revelaram-se mais apropriadas para imagens de elevada dimensão. A abordagem baseada em fila (`ImageRegionFillingWithQUEUE`) demonstrou ser particularmente robusta, mitigando o risco de sobrecarga da *stack* (*stack overflow*) e mantendo uma ordem de preenchimento regular e eficiente. Consequentemente, para aplicações que envolvam a segmentação de regiões extensas, recomenda-se a utilização da versão baseada em fila, garantindo uma maior fiabilidade e desempenho consistente.